



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Company Tutors:

Dr. David Soldani

Dr. Vincenzo Cosi

Academic Year 2025-2026

This thesis presents the design and implementation of a security monitoring tool that collects and analyzes process-related information directly from the operating system kernel using eBPF (extended Berkeley Packet Filter). By leveraging kernel-level instrumentation, the system captures detailed process events with high accuracy and minimal overhead, overcoming the limitations of traditional user-space monitoring approaches.

The proposed tool adopts a modular and efficient architecture to ensure scalability and real-time data collection. The gathered information is structured to support security analysis, enabling the detection of anomalous behaviors such as unauthorized executions and privilege escalations.

This work demonstrates how kernel-derived process data, combined with eBPF technology, can enhance system visibility and provide a solid foundation for advanced security monitoring and threat detection.

Acknowledgements

Only in the printed version

Contents

Acronyms	1
1 Introduction	2
1.1 Context and Motivation	2
1.2 Problem Statement	3
1.3 Objectives and Thesis Structure	5
2 Background and eBPF architecture	7
2.1 Linux Kernel Observability	7
2.2 eBPF Fundamentals	11
2.2.1 Execution Model, Verifier and JIT Compilation	11
2.2.2 Hook Selection and Event Semantics	13
2.2.3 Maps, Helpers and Kernel-to-Userspace Transport	14
2.2.4 BTF and CO-RE Portability	16
2.3 Runtime Security and Anomaly Detection	17
2.3.1 Point, Contextual and Collective Anomalies	17
2.3.2 Policies, Detectors and Alert Generation	18
2.3.3 MITRE ATT&CK as a Classification Framework	19
2.4 Related Runtime Security Tools	19
2.4.1 Tracee	20
2.4.2 Other Relevant eBPF Security Tools	20

CONTENTS

2.5 Positioning of the Proposed Work	21
A Appendix A	23
B Appendix B	24
Bibliography	25

List of Figures

- 2.1 High-level eBPF execution workflow 12
- 2.2 Complementary evidence exposed by different observation points
along a kernel operation. 14
- 2.3 Anomaly Detection: Type of Anomalies 17

List of Tables

2.1	Observation sources for eBPF runtime monitoring	10
2.2	Comparison of eBPF event-transport mechanisms	15

Acronyms

eBPF	extended Berkeley Packet Filter
CPU	central processing unit
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
LSM	Linux Security Module
JIT	just-in-time
BTF	BPF Type Format
CO-RE	Compile Once - Run Everywhere

Chapter 1

Introduction

1.1 Context and Motivation

Modern Linux systems execute applications, containers, and infrastructure services whose security-relevant behaviour changes continuously at runtime. Static inspection can describe software before execution, while application logs expose only the information that applications choose to report. Neither source alone provides a complete account of operations such as process creation, credential changes, file access, memory protection changes, namespace activity, or interactions with kernel security mechanisms. Runtime security monitoring therefore requires an observation point that is close enough to the operating system to capture these actions as they occur.

The Linux kernel is a valuable observation layer because it mediates processes, files, credentials, memory, and other shared resources. Kernel-level telemetry can reveal behaviour that is distributed across several applications or omitted from their logs. This visibility is particularly relevant in containerised environments, where workloads remain isolated at the application level but share the host kernel. At the same time, kernel instrumentation is security-sensitive: excessive privileges or unsafe extensions may increase the attack surface, especially across container boundaries [14]. A monitoring design must consequently balance visibility, compatibility, operational cost, and the risks associated with privileged execution.

Runtime security tools increasingly use eBPF to obtain this visibility. Existing systems demonstrate that eBPF can support process, file, network, and security telemetry in cloud-native environments, although their archi-

textures and performance characteristics differ [15]. This thesis focuses on the process and host-security side of that design space.

eBPF provides a controlled runtime inside the Linux kernel. User-space software loads constrained programs that can be attached to supported processes, networking, and security hooks. Before a program is accepted, the kernel verifier analyses its control flow and use of memory, registers, pointers, and helper functions through static analysis. Accepted programs can then collect or transform data when the corresponding hook is reached [12, 16, 17].

This execution model is well suited to runtime monitoring for three reasons. First, collection occurs at the point where the kernel processes the operation, reducing dependence on application-specific instrumentation. Second, eBPF maps and event buffers provide structured communication between kernel and user space. Third, programs can reject irrelevant activity before transporting it to user space, which creates an opportunity to control processing overhead. These properties do not make an eBPF tool automatically safe, portable, or efficient: verifier behaviour, available program types, helper functions, kernel data structures, and attachment mechanisms vary across kernel versions. They instead provide the foundation on which a target-aware monitoring pipeline can be built.

The work was developed in an industrial collaboration and targets Rocky Linux 8.10 with the kernel `4.18.0-553.109.1.el8_10.x86_64`. This environment is important because it combines a long-lived enterprise kernel line with backported functionality. A feature exposed by a newer upstream kernel cannot be assumed to exist, or to behave identically, on this target [22]. Compatibility is therefore treated as an architectural constraint.

Within this target environment, the prototype is designed as a host-level runtime monitor. It collects and processes security-relevant events on the monitored system and performs bounded local detection over individual events and short event sequences.

1.2 Problem Statement

The central problem addressed by this thesis is how to convert heterogeneous kernel observations into useful and readable security information on a constrained enterprise Linux target. Capturing an event is only the first step. A practical pipeline must assign a stable identity to the event, transport it

without confusing records produced by different probes, decode typed arguments, apply user-selected scope, and present the result consistently. When these contracts diverge, valid kernel records may be decoded as unknown events, attributed to the wrong event type, or silently discarded.

Individual observations also have different analytical value. Some are relevant in isolation, for example an access to a sensitive file under narrowly defined conditions. Others become relevant only as an ordered group, such as a credential transition followed shortly by process execution or sensitive file access. Following the established anomaly-detection taxonomy, the thesis uses *point event* for a locally significant observation and *short collective event pattern* for a bounded sequence whose combined meaning is more relevant than its elements considered independently [9]. These terms describe deterministic, rule-based detection; they do not imply machine learning, statistical baselines, or unsupervised anomaly detection methods.

Collecting kernel events alone does not make the resulting telemetry immediately useful for security analysis. The proposed system therefore introduces a configurable policy and detection layer that transforms selected runtime observations into focused security information. Policies define the monitoring scope by selecting the event domain relevant to a particular scenario, helping to limit noise and unnecessary processing. Detectors operate on the resulting normalised event stream and encode reusable conditions that can generate an alert from either an individual event or a short process-aware sequence. This separation allows collection scope and detection logic to evolve independently, without requiring each analytical change to be implemented inside the eBPF programs. The generated alerts are enriched with metadata based on MITRE ATT&CK [24], providing a standard vocabulary for interpreting the observed behaviour and supporting downstream analysis.

Four constraints shape the problem. First, the target kernel limits which attachment mechanisms, helpers, maps, and portability techniques can be used reliably. Second, the kernel verifier constrains program complexity and memory access, so event enrichment must be divided carefully between kernel and user space. Third, high-frequency hooks can generate enough data to make transport, decoding, rule evaluation, and output formatting more expensive than the probe itself. Fourth, the agent is expected to run beside production workloads; CPU and memory use are therefore evaluation criteria rather than secondary implementation details.

These constraints motivate a selective architecture. The proposed tool uses eBPF for collection and optional early filtering, while user space owns event

normalisation, policy management, detector evaluation, alert generation, and presentation. The user-space runtime is implemented in Go, which was selected in part because libraries such as `libbpfgo` provide Go bindings to `libbpf` for loading eBPF objects and managing programs, maps, and event buffers [2]. The design preserves a stable kernel-to-user-space contract while allowing detection logic to evolve without recompiling every eBPF program.

1.3 Objectives and Thesis Structure

The main objective of this thesis is to design, implement, and evaluate an eBPF-based runtime security monitoring prototype that collects process and security-relevant kernel events on the target Rocky Linux environment, normalises them in user space, and evaluates them through configurable policies and local detectors.

The remainder of the thesis develops this objective progressively, moving from the necessary technical foundations to the architecture, implementation, and evaluation of the proposed system:

- **Chapter 2 - Background and eBPF architecture** introduces Linux runtime instrumentation, the eBPF execution model, and the concepts required to understand process and security monitoring. It also reviews related runtime-security tools and policy-driven detection approaches.
- **Chapter 3 - Requirements and System Design** defines the target environment, functional and operational requirements, architectural components, event flow, and the boundaries between kernel-space collection and user-space processing.
- **Chapter 4 - Implementation** describes the eBPF programs, probe management, binary event contract, Go runtime, decoder, policy and detector subsystems, MITRE ATT&CK metadata, output handling, logging, and container packaging.
- **Chapter 5 - Experimental Evaluation** evaluates functional behaviour, event and detector operation, CPU and memory consumption, and the effects of the implemented filtering and performance controls on the target Rocky Linux environment.

- **Chapter 6 - Conclusions** summarises the results, discusses the principal limitations of the prototype, and identifies possible directions for future development.

Chapter 2

Background and eBPF architecture

2.1 Linux Kernel Observability

Runtime monitoring requires evidence about operations while they are being executed. Application logs describe domain-specific actions, but their content depends on what each application chooses to record. The Linux kernel offers a complementary observation layer because it mediates process creation, credential changes, file access, memory management, inter-process communication, and other security-relevant operations. Kernel observability is therefore useful when behaviour crosses application boundaries or is not represented consistently in application logs [41, 12].

Linux does not expose all runtime activity through one universal event source. Instead, it provides instrumentation points at different layers of kernel execution. Each mechanism determines when monitoring code runs, which context is available, and how strongly the observation depends on kernel internals.

System-call observation. A system call is the controlled interface through which a user-space process requests a service from the kernel. It is not itself an instrumentation mechanism. Monitoring systems commonly observe this boundary through system-call tracepoints, such as entry and exit events. An entry event exposes the requested operation and its original arguments, whereas an exit event exposes the returned result. Correlating the two can

distinguish an attempted operation from one that completed successfully. This view is broad and process-oriented, but it remains close to the user-space request: one system call may traverse several internal functions, and its original arguments may not identify the final kernel object or security decision [44, 41].

Tracepoints and raw tracepoints. A tracepoint is a static observation point deliberately inserted into the kernel source by kernel developers. It has a name, belongs to an event category, and defines the data made available when that point is reached. A tracepoint remains inactive until a tracing component is attached to it. In an eBPF-based monitor, this component is an eBPF program loaded from user space and attached to the selected tracepoint. Each time kernel execution reaches that point, the kernel invokes the attached program and provides a tracepoint-specific context containing the declared event fields.

A regular tracepoint gives the eBPF program a structured event context whose layout follows the format exported for that tracepoint. A raw tracepoint refers to the same underlying kernel instrumentation site, but exposes the original arguments passed by the kernel to that site with less intermediate transformation. This reduces preparation performed before the eBPF program runs, while requiring the program to know how each raw argument must be interpreted. Tracepoints are therefore explicit and comparatively stable observation points, whereas raw tracepoints trade some of that abstraction for more direct access to the underlying arguments. Both mechanisms are limited to the tracepoints actually provided by the target kernel [44, 43].

Kprobes and kretprobes. A kprobe is a dynamic probe placed at a selected kernel instruction at run time. Monitoring tools usually identify that instruction through the symbol of a kernel function and place the probe at its entry. Unlike a tracepoint, the observation point does not need to have been prepared explicitly by the kernel developer. When execution reaches the probed instruction, the kernel temporarily transfers control to the probing infrastructure, which invokes the attached eBPF program before normal execution continues. The program receives a snapshot of the processor-register state. Function arguments are not presented as a tracepoint-specific event structure; they must be recovered from the appropriate registers according to the architecture and calling convention.

A kretprobe observes the corresponding function-return path. It causes an

attached program to run when an invocation of the probed function finishes and makes the return value available through the register context. A kprobe can therefore describe the operation requested at function entry, while a kretprobe can reveal the result produced at function exit. Entry and return records are separate observations: when both the original arguments and the result are required in one logical event, the monitoring system must save the entry data and correlate it with the return observation, commonly by using the current thread identity.

Kprobes and kretprobes provide access to functions for which no tracepoint exists, but this flexibility increases coupling to kernel implementation details. A renamed or inlined function, a changed signature, a different calling convention, or an unavailable symbol can invalidate the attachment or the argument-decoding logic. They are therefore powerful dynamic mechanisms, but generally require more target-specific validation than tracepoints [38].

Linux Security Module hooks. The LSM framework inserts security mediation points into kernel operations involving files, credentials, processes, sockets, and other protected objects. At these points, registered security modules can inspect an operation and, for hooks with authorisation semantics, permit or reject it. Observing an LSM-related function can therefore expose a resolved kernel object and the security-relevant stage of an operation, rather than only the original system-call request. Two concepts should not be confused: an LSM hook is the kernel mediation point, while an eBPF LSM program is one possible program type for attaching logic to supported hooks. On kernels where that program type is unavailable or unsuitable, monitoring tools may instead observe the corresponding security function through a kprobe. An entry-side observation still does not necessarily reveal the final outcome; when the result matters, it must be obtained from a compatible return-side observation [40, 43, 38].

Observation source	Information exposed	Main trade-off
System-call tracepoint	Arguments at entry or return value at exit.	Broad process visibility, but limited insight into internal kernel processing and resolved objects.
Kernel tracepoint	Structured fields defined for a specific kernel event.	Clear semantics and simple decoding, but only where a suitable tracepoint is available.
Raw tracepoint	Original tracepoint arguments with minimal preprocessing.	More direct access, but requires kernel-aware argument decoding.
Kprobe / kretprobe	Register state at function entry or return.	Flexible placement, but stronger dependence on kernel symbols, signatures and calling conventions.
LSM-related hook	Security context, resolved objects and mediation stage.	Rich security semantics, but attachment and result visibility depend on target-kernel support.

Table 2.1: Comparison of the principal observation sources relevant to eBPF-based runtime monitoring [44, 38, 40, 43].

The role of eBPF. eBPF is the execution facility attached to an observation point, not a replacement name for the observation point itself. The selected hook causes an eBPF program to run and determines the context received by that program. The program can then extract selected fields, apply an early filter, update maps, or submit an event to user space. Consequently, “tracepoint”, “kprobe”, “LSM hook”, and “eBPF program” describe different parts of the same monitoring path [16, 43].

These mechanisms are complementary. System-call describe requests and results at the user–kernel boundary; general tracepoints expose kernel-defined events; kprobes and kretprobes provide dynamic access to function entry and return; and LSM-related hooks expose security mediation points. A runtime monitor combines them according to the required semantics, available target-kernel features, and acceptable coupling to kernel internals. The engineering objective is to select observation points that provide sufficient and reliable evidence for the intended security analysis [41, 15].

2.2 eBPF Fundamentals

eBPF is a modern technology that makes the Linux kernel programmable at runtime. It allows small programs to be loaded from user space and executed inside the kernel when a selected event occurs. Before execution, the kernel verifies that each program respects the safety constraints associated with its program type and intended attachment point [25, 16].

This approach makes it possible to extend kernel behaviour without modifying the kernel source or developing a traditional kernel module for each new functionality. eBPF programs can be loaded, replaced, and specialised for different purposes, including system observability, performance analysis, network processing, and security. Its principal benefit is therefore controlled programmability: functionality can be introduced close to the kernel operations of interest while remaining subject to verification and well-defined execution constraints [12, 17].

2.2.1 Execution Model, Verifier and JIT Compilation

An eBPF program is compiled into instructions for the BPF virtual machine and submitted to the kernel through the BPF user-space interface. The instruction set defines general-purpose registers, a bounded stack, arithmetic and branch instructions, memory operations, and calls to kernel-provided functions. The program does not execute as unrestricted kernel code: its available context, memory accesses, and callable functions depend on its program type [25, 18].

Because the programs described above are loaded from user space but execute inside the kernel, they cannot be allowed to run without first establishing that they respect the kernel’s safety constraints. An unsafe memory access or an invalid operation in this context could affect the stability and security of the entire system. Linux addresses this requirement through the eBPF verifier, the kernel component responsible for deciding whether a submitted program can be accepted for execution. Rather than trusting the program, the verifier analyses it during the loading process and before permitting its execution; a program that does not satisfy the required safety conditions is rejected and never executed.

The verifier performs a path-sensitive static analysis of the program’s control flow. It follows the possible execution paths while tracking the state of registers and stack locations, distinguishing scalar values from different

classes of pointers, checking memory bounds and alignment, and validating the arguments supplied to helper functions. Paths that may access invalid memory, use uninitialised values, or leave kernel-managed resources unreleased are rejected. To avoid analysing equivalent situations repeatedly, the verifier uses state pruning: when a previously accepted verifier state already covers a new state reached at the same instruction, the redundant path does not need to be explored again [17, 13].

Verification is a necessary safety gate, not a proof that a program implements the intended monitoring semantics. A verifier-accepted program may still read the wrong field, emit an incomplete event, or encode data inconsistently with its user-space decoder. Conversely, semantically reasonable source code may be rejected because the verifier cannot establish the required safety properties for a particular kernel. Verifier behaviour is therefore both a security mechanism and a practical compatibility constraint [17, 31].

Once the verifier accepts a program, the kernel may pass its eBPF instructions to an architecture-specific JIT compiler. The compiler translates the verified eBPF bytecode into native machine instructions for the host architecture. The resulting native code can then be executed directly whenever the program is invoked, avoiding the overhead of processing the eBPF instructions individually at each execution [36, 37].

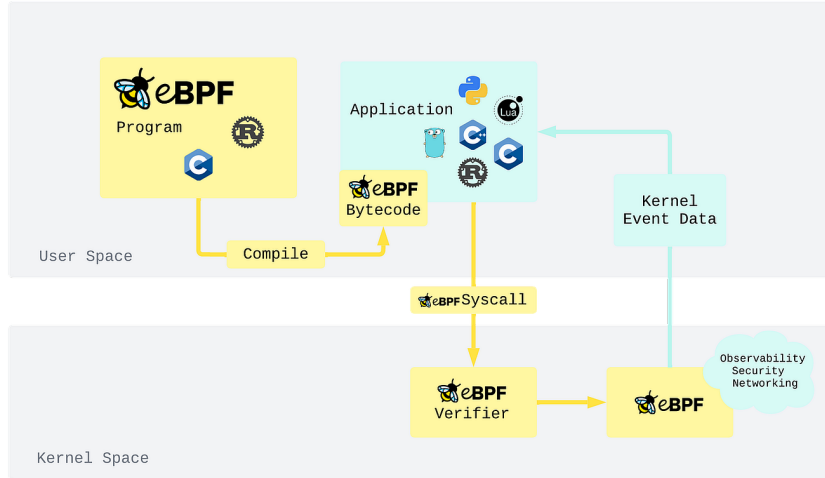


Figure 2.1: High-level eBPF execution workflow.

JIT compilation does not alter the functionality authorised by the verifier.

The translation is performed only after verification and preserves the semantics of the accepted eBPF program. Its availability depends on the host architecture, the kernel build configuration, and the runtime JIT settings.

This completes the basic eBPF execution path considered here. A program is compiled into eBPF instructions, submitted to the kernel, analysed by the verifier and, if accepted, translated into native machine code and executed whenever its associated hook or execution context is invoked [12].

2.2.2 Hook Selection and Event Semantics

An eBPF program type defines the execution contract enforced by the kernel, including the expected context, available helpers, and permitted return semantics. The hook instead identifies the event or kernel execution point that triggers the program, while the attachment mechanism binds the loaded program to a compatible hook. These concepts are related, but they are not interchangeable [43, 18].

The selected observation point determines not only when an eBPF program runs, but also the meaning of the resulting event. A system-call entry hook captures the operation requested by a process and its original arguments. A later tracepoint, dynamic probe, or LSM-related hook may expose kernel-resolved objects or the security mediation stage of the operation. An exit or return-side observation can instead reveal whether the operation completed successfully [44, 38, 40].

These observations represent different stages of the same kernel execution path and should therefore be considered complementary rather than interchangeable. No single hook necessarily exposes the original request, the resolved kernel context, and the final result. A runtime monitor may consequently combine multiple event sources and correlate their records through process identity and temporal proximity.

Hook selection also introduces a compatibility trade-off. Explicit tracepoints provide structured interfaces when the target kernel exports a suitable event, whereas dynamic probes offer broader placement flexibility but depend more strongly on kernel symbols, function signatures, and calling conventions. The appropriate observation point must therefore balance the required security semantics with the capabilities and stability of the deployment kernel [44, 38, 22].

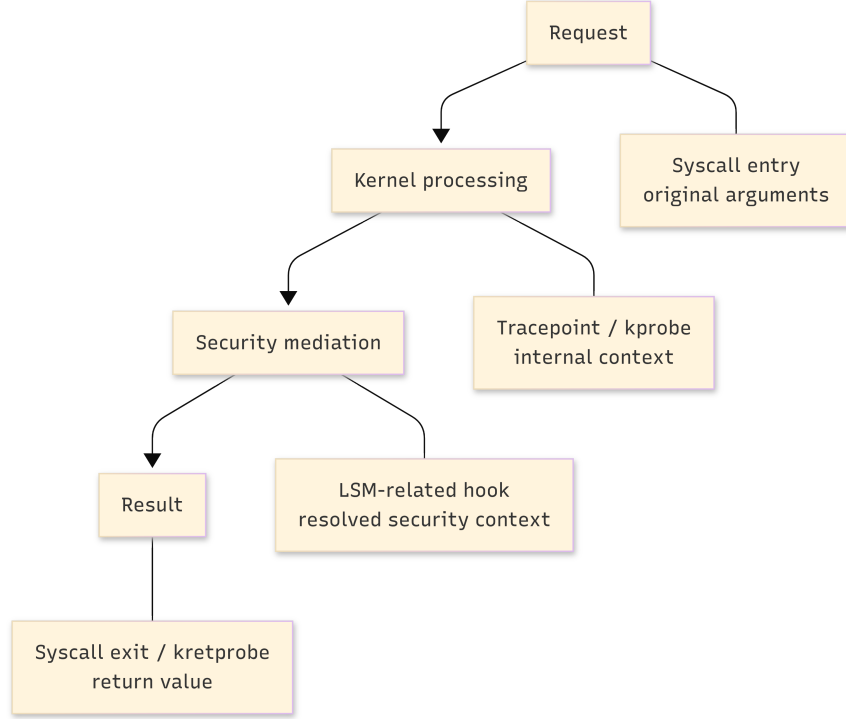


Figure 2.2: Complementary evidence exposed by different observation points along a kernel operation.

2.2.3 Maps, Helpers and Kernel-to-Userspace Transport

The execution of an eBPF program is intentionally constrained and usually short-lived. Runtime monitoring, however, requires information to persist beyond a single invocation and eventually reach user space, where more complex processing can be performed. Linux supports this interaction through kernel-managed state and a controlled communication path between eBPF programs and the user-space runtime. Maps, helpers, and event buffers form the main building blocks of this path [33, 18, 42, 34].

Maps are typed kernel objects that allow eBPF programs to preserve state across invocations and exchange data with user space. Each map type defines its own access and concurrency semantics, while its structure and capacity are established at creation time. By moving persistent configuration and temporary state outside the constrained BPF stack, maps extend the

information that a program can retain between executions. Accesses performed by eBPF code remain subject to verifier validation and to the rules imposed by the selected map type [33, 19].

Helpers are kernel-provided functions that extend what an eBPF program can do without granting unrestricted access to kernel internals. They expose selected services through a controlled interface, allowing the kernel to validate how those services are invoked and what data is passed to them. The set of available helpers is determined by the program type and by the capabilities of the target kernel, so support documented for eBPF in general does not imply that the same helper can be used in every execution context. This constraint is particularly relevant to this work, which targets Rocky Linux 8.10 with kernel 4.18.0-553.109.1.el8_10.x86_64: helper selection must therefore be validated against the capabilities of the deployed kernel and the program type used by each probe, rather than inferred solely from current upstream documentation [18, 17].

Event transport carries records produced during eBPF execution to user space, where they can be decoded and processed without extending the work performed inside the kernel. The BPF ring buffer provides a shared multi-producer, single-consumer channel designed to use buffer capacity across CPUs and preserve the order in which records are reserved. Perf-event transport follows a different model: a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` directs records to per-CPU perf ring buffers, which are consumed independently and can report lost events.

Both mechanisms rely on finite buffer capacity. When kernel-side producers generate data faster than user space can consume it, a submission may fail or a record may be lost. Event transport must therefore be treated as part of the monitor’s correctness model rather than as a transparent delivery channel [34, 42, 18].

Mechanism	Buffer organisation	Main implication
BPF ring buffer	Shared multi-producer, single-consumer buffer	Shares capacity across CPUs and preserves reservation order between producers.
Perf-event transport	Per-CPU buffers referenced through a perf-event array	Buffers are consumed independently and provide explicit lost-event notifications.

Table 2.2: Comparison of eBPF event-transport mechanisms [34, 42].

Regardless of the selected transport mechanism, kernel and user space must share a consistent record format so that the transferred data can be decoded into correctly typed events [12].

2.2.4 BTF and CO-RE Portability

An eBPF program often interacts with kernel data structures whose layout may change across kernel versions and distribution builds. A program compiled against one specific set of kernel headers may therefore rely on field offsets or type definitions that do not match the target system. Portability requires a way to preserve the meaning of those accesses while adapting them to the types exposed by the deployed kernel. BTF and CO-RE provide the metadata and relocation mechanisms used to address this problem.

BTF represents types and related metadata in a compact format understood by the kernel and BPF tooling. BTF may describe C types and strings and can also carry function, line, and relocation information. When kernel BTF is available, loaders and debugging tools can reason about kernel data structures without depending solely on a set of headers copied from one kernel build [35].

CO-RE uses BTF information to adapt an eBPF program to the kernel on which it is loaded. During compilation, Clang stores relocation records that describe the kernel types and fields accessed by the program. At load time, `libbpf` reads the BTF information of the running kernel and updates the relevant eBPF instructions so that they use the actual layout of those structures. As a result, the same compiled object can continue to work when compatible kernel versions place a field at a different offset, without requiring the program to be recompiled on the monitored system [35, 32, 39].

In this thesis, this mechanism helps the eBPF programs access kernel structures according to the layout exposed by Rocky Linux 8.10 with kernel `4.18.0-553.109.1.el8_10.x86_64`. However, CO-RE only adapts compatible differences in type and data-structure layout. It cannot make a missing hook, helper, program type, or attachment mechanism available, and it does not ensure that the target verifier will accept the program. These capabilities must therefore be checked directly on the deployed kernel. This is especially important for enterprise distributions, where backported features may make the effective kernel capabilities differ from those suggested by the upstream version number alone [31, 22].

2.3 Runtime Security and Anomaly Detection

Kernel telemetry becomes useful for runtime security only when observed operations are interpreted in relation to a security condition. Detection systems can assign this meaning in different ways. Signature-based detection matches observed activity against predefined patterns, whereas anomaly-based detection identifies deviations from an expected model of behaviour [23, 9].

The detector layer developed in this thesis follows a deterministic, rule-based approach. It evaluates explicit conditions over individual events or short ordered event sequences and does not learn a behavioural baseline or calculate statistical anomaly scores. The point and collective taxonomy introduced below is therefore used to describe the structure of the observations involved in a detection, rather than the algorithm through which the system discovers anomalous behaviour.

2.3.1 Point, Contextual and Collective Anomalies

Chandola, Banerjee, and Kumar distinguish point, contextual, and collective anomalies. A point anomaly is an individual observation that is anomalous with respect to the remaining data. A contextual anomaly is anomalous only under particular contextual attributes, such as time or location. A collective anomaly is a related group of observations whose joint behaviour is anomalous, even when the individual observations may not be unusual [9].

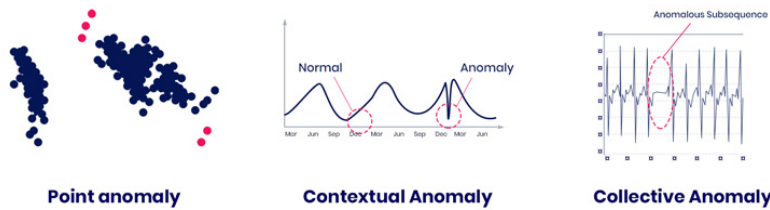


Figure 2.3: Anomaly Detection, Type of Anomalies.[11]

This taxonomy is useful for kernel telemetry. A single operation can be security-relevant when its own attributes satisfy a sufficiently precise rule.

Other behaviours are better represented as an ordered group, for example a credential transition followed shortly by execution or access to a sensitive resource. Event-stream research formalises related ideas through ordered patterns, selection strategies, and bounded temporal constraints [10, 1].

Contextual detection depends on understanding the circumstances in which an event occurs. A monitor running on a single host can interpret an observation using the process information available at that moment, but its view remains limited to the local system. It cannot reliably determine whether the same behaviour is unusual across a wider deployment or inconsistent with organisational expectations that are not visible to the agent. For this reason, the tool developed in this work focuses on deterministic detections over individual events and short, locally correlated sequences. It does not attempt to provide contextual anomaly detection or general-purpose complex event processing.

2.3.2 Policies, Detectors and Alert Generation

Kernel telemetry becomes useful for security analysis only after the collected observations have been placed within a defined monitoring context and evaluated against explicit conditions. The tool developed in this work follows this principle by separating the selection of relevant activity from its security interpretation. Policies define the monitoring scope by selecting the events and process contexts admitted to the analysis pipeline, while detectors operate on the resulting normalised stream and evaluate explicit security conditions. Keeping these responsibilities separate allows the monitoring scope and the detection logic to evolve independently, without moving higher-level analysis into the eBPF programs. Comparable separations between event selection and rule evaluation appear in established runtime security systems [8, 4, 30].

A point detector evaluates one event and emits an alert when its predicates match. A collective detector additionally needs state: it must retain enough information to decide whether later events complete a potentially malicious pattern. Correlation keys constrain which observations may belong to the same sequence, and a time window bounds how long partial matches remain relevant. These choices reduce accidental correlation between unrelated processes, although they do not by themselves prove that every emitted alert is malicious [10, 1].

An alert is a higher-level result derived from one or more source events. A

useful alert preserves enough provenance for an operator or downstream system to understand the rule that matched and the evidence that triggered it. Typical fields include detector identity, severity, source process attributes, the relevant event arguments, and, for collective matches, the ordered event sequence. This separation also permits raw telemetry and alert output to be selected independently according to operational needs [23, 28].

2.3.3 MITRE ATT&CK as a Classification Framework

MITRE ATT&CK organises observed adversary behaviour into tactics and techniques. Tactics express the adversary’s objective, while techniques describe ways in which that objective may be pursued. The framework provides a shared vocabulary for describing behaviour across detections, reports, and security products [24, 20].

Associating detector output with ATT&CK metadata improves the interpretability and interoperability of alerts. It enables an analyst to relate a local rule to a recognised behavioural category and allows downstream systems to group or search alerts using stable identifiers. In the proposed work, the mapping is therefore treated as part of the detector contract rather than as decorative output. Its purpose is to make the security meaning of a match explicit and to support structured analysis across the detector set [24, 20].

The framework remains a classification system rather than an executable detection language. A technique can manifest through different event sequences, and one local observation may support several interpretations. Detector-to-technique mappings consequently require deliberate review against the behaviour represented by each rule [24, 21].

2.4 Related Runtime Security Tools

Open-source runtime security tools use kernel telemetry in materially different architectures. Tracee presents a broad event pipeline with policy selection and detectors; Falco centres its design on declarative rules over event streams; Tetragon combines observability with hook-specific selectors and optional in-kernel actions. They are useful architectural references, but their event catalogues, threat models, and operational goals are not interchangeable [15].

2.4.1 Tracee

Tracee is a Linux runtime security and observability system that represents system calls, process lifecycle activity, file operations, networking data, LSM-related observations, and derived security detections through a common event abstraction. Events are grouped into categories and can be selected individually or through sets [5, 6].

Tracee policies combine scope with rules that select the events and data of interest. Filtering is divided between kernel and user space according to the available event data and filter type. This architecture can avoid submitting some irrelevant records while retaining richer processing after events reach user space [8]. Detection remains a separate layer: current detectors declare their input events, process matching observations, and can emit derived events. Tracee supports detector implementations in Go and declarative YAML definitions, and detector output can participate in the same event-oriented pipeline [4, 3].

The system supports structured and human-readable output, stream routing, and enrichment for operational use. Its breadth and mature event model make it an important reference for separating collection, selection, detection, and presentation [7]. The proposed work follows that separation as a design pattern, but it is an independent and narrower research prototype. It does not claim Tracee’s event coverage, detector ecosystem, forensic facilities, or functional equivalence.

2.4.2 Other Relevant eBPF Security Tools

Falco is a runtime security engine organised around rules evaluated over event sources. Kernel system-call activity can be captured through its driver architecture, while plugins provide additional sources. Falco rules are declarative YAML objects containing a condition, output description, priority, and optional reusable lists, macros, tags, and exceptions. A matching rule produces an alert that can be routed to configured outputs [27, 30, 28].

This model demonstrates the operational value of readable rules, reusable expressions, explicit exceptions, and an alert-oriented interface. Falco also maintains process and container context used to enrich individual events. However, its documented rule conditions are evaluated against an incoming event, and separate event sources are processed independently; the core rule language is not presented as a general temporal sequence language across

heterogeneous sources [26, 29].

Tetragon combines eBPF-based observability with optional runtime enforcement. Its tracing policies specify kernel hooks, captured arguments or return values, and selectors that can match process, namespace, capability, workload, and argument attributes. Supported actions can include emitting an event, delivering a signal, or overriding a return value where the hook and kernel support that operation [48, 49, 45].

Tetragon therefore moves more selection and potential response close to the hook than an architecture centred on user-space detectors. It preserves process-lifecycle context and emits structured events through its own interfaces, but its documented tracing-policy model is principally hook-centred rather than a general language for arbitrary temporal sequences [46, 48]. Its current installation requirements also illustrate the dependence of advanced eBPF functionality on kernel capabilities and BTF availability [47].

Together, Falco and Tetragon mark two useful boundaries for the proposed work. Falco shows a mature single-event rule and alert model, while Tetragon shows the advantages and platform requirements of in-kernel selectors and enforcement. The prototype considered in this thesis instead retains enforcement outside its scope and concentrates on local observation, normalisation, deterministic point rules, and short process-aware collective patterns.

2.5 Positioning of the Proposed Work

The proposed work occupies a deliberately bounded part of the Linux runtime security design space. It targets process and host-security monitoring on Rocky Linux 8.10 and its enterprise 4.18-based kernel. Vendor backports and target-specific verifier behaviour make this environment more nuanced than its upstream version number suggests, so compatibility is treated as a property to be established on the target rather than inferred from nominal kernel age [22].

At a high level, eBPF programs collect selected kernel observations and can apply a minimal early filter, while a Go runtime normalises the resulting event stream. Declarative policies define the event domain, and automatically activated YAML detectors evaluate deterministic predicates over individual events or short ordered patterns. Collective detection is intentionally local and bounded: it uses process identity and immediate parent informa-

tion rather than a persistent, cluster-wide process graph. Alerts preserve their source evidence and carry ATT&CK metadata as a common classification vocabulary.

This combination should be understood as an integration and engineering contribution rather than a claim that its individual mechanisms are unprecedented. Kernel event collection, declarative rules, policy-based selection, user-space state, and ATT&CK-labelled alerts all have established precedents in Tracee, Falco, Tetragon, and related systems [8, 4, 30, 48]. The thesis instead examines how these ideas can be assembled into a compact, independently implemented pipeline under the constraints of the selected enterprise kernel.

Several boundaries remain explicit. The system is an observation and detection prototype, not an enforcement engine. Its rules are deterministic rather than machine-learned, its collective model covers short local sequences rather than general contextual or cluster-wide correlation, and its implemented event families represent selected coverage rather than complete host visibility. Performance, compatibility, detector behaviour, and the effect of filtering are evaluated experimentally in Chapter 5 rather than assumed from the architecture. Possible orchestration in a wider deployment remains future context and is not treated as a contribution of this thesis.

Appendix A

Appendix A

Appendix text here.

Appendix B

Appendix B

Appendix text here.

Bibliography

- [1] Jagrati Agrawal et al. “Efficient Pattern Matching over Event Streams”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. Ed. by Jason Tsong-Li Wang. New York, NY, USA: Association for Computing Machinery, 2008, pp. 147–160. ISBN: 978-1-60558-102-6. DOI: [10.1145/1376616.1376634](https://doi.org/10.1145/1376616.1376634).
- [2] Aqua Security. *libbpfgo: An eBPF Library for Go Powered by libbpf*. Official project repository. 2026. URL: <https://github.com/aquasecurity/libbpfgo> (visited on 07/27/2026).
- [3] Aqua Security. *Tracee Detector API Reference*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/api-reference/> (visited on 07/27/2026).
- [4] Aqua Security. *Tracee Detectors*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/> (visited on 07/27/2026).
- [5] Aqua Security. *Tracee Documentation*. 2026. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).
- [6] Aqua Security. *Tracee Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/> (visited on 07/27/2026).
- [7] Aqua Security. *Tracee Output Configuration*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/outputs/> (visited on 07/27/2026).
- [8] Aqua Security. *Tracee Policies: Overview*. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).

BIBLIOGRAPHY

- [9] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (July 2009), 15:1–15:58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [10] Gianpaolo Cugola and Alessandro Margara. “Processing Flows of Information: From Data Stream to Complex Event Processing”. In: *ACM Computing Surveys* 44.3 (June 2012), 15:1–15:62. DOI: [10.1145/2187671.2187677](https://doi.org/10.1145/2187671.2187677).
- [11] Einar & Partners Research Unit. *Mystery of Predictions in Anomaly Detection*. Einar & Partners Research Unit. URL: <https://research.einar.partners/mystery-of-predictions-in-anomaly-detection/> (visited on 07/28/2026).
- [12] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.
- [13] Elazar Gershuni et al. “Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Ed. by Kathryn S. McKinley and Kathleen Fisher. PLDI ’19. Association for Computing Machinery, 2019, pp. 1069–1084. DOI: [10.1145/3314221.3314590](https://doi.org/10.1145/3314221.3314590). URL: <https://doi.org/10.1145/3314221.3314590>.
- [14] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Ed. by Joseph A. Calandrino and Carmela Troncoso. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5971–5988. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [15] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. DOI: [10.1109/ACCESS.2025.3605432](https://doi.org/10.1109/ACCESS.2025.3605432). URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [16] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. 2024. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).
- [17] Linux Kernel Community. *eBPF Verifier*. 2026. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).

BIBLIOGRAPHY

- [18] Linux man-pages project. *bpf-helpers(7) — List of eBPF Helper Functions*. 2026. URL: <https://man7.org/linux/man-pages/man7/bpf-helpers.7.html> (visited on 07/27/2026).
- [19] Linux man-pages project. *bpf(2) — Perform a Command on an Extended BPF Map or Program*. 2026. URL: <https://man7.org/linux/man-pages/man2/bpf.2.html> (visited on 07/27/2026).
- [20] MITRE Corporation. *Get Started with MITRE ATT&CK*. 2026. URL: <https://attack.mitre.org/resources/> (visited on 07/27/2026).
- [21] MITRE Corporation. *MITRE ATT&CK Frequently Asked Questions*. 2026. URL: <https://attack.mitre.org/resources/faq/> (visited on 07/27/2026).
- [22] Red Hat. *What Is Backporting and How Does It Affect Red Hat Enterprise Linux?* Red Hat Customer Portal. June 2025. URL: <https://access.redhat.com/solutions/57665> (visited on 07/27/2026).
- [23] Karen Scarfone and Peter Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Special Publication 800-94. National Institute of Standards and Technology, Feb. 2007. DOI: [10.6028/NIST.SP.800-94](https://doi.org/10.6028/NIST.SP.800-94). URL: <https://doi.org/10.6028/NIST.SP.800-94>.
- [24] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [25] Dave Thaler. *BPF Instruction Set Architecture (ISA)*. RFC 9669. Internet Engineering Task Force, Oct. 2024. DOI: [10.17487/RFC9669](https://doi.org/10.17487/RFC9669). URL: <https://www.rfc-editor.org/info/rfc9669>.
- [26] The Falco Project Authors. *Falco Event Sources*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/> (visited on 07/27/2026).
- [27] The Falco Project Authors. *Falco Kernel Event Architecture*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/kernel/architecture/> (visited on 07/27/2026).
- [28] The Falco Project Authors. *Falco Outputs*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/outputs/> (visited on 07/27/2026).
- [29] The Falco Project Authors. *Falco Rule Conditions*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/conditions/> (visited on 07/27/2026).

BIBLIOGRAPHY

- [30] The Falco Project Authors. *Falco Rules*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/> (visited on 07/27/2026).
- [31] The Linux Kernel Documentation. *BPF Design Q&A*. 2026. URL: https://docs.kernel.org/bpf/bpf_design_QA.html (visited on 07/27/2026).
- [32] The Linux Kernel Documentation. *BPF LLVM Relocations*. 2026. URL: https://docs.kernel.org/bpf/llvm_reloc.html (visited on 07/27/2026).
- [33] The Linux Kernel Documentation. *BPF Maps*. 2026. URL: <https://docs.kernel.org/bpf/maps.html> (visited on 07/27/2026).
- [34] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 07/27/2026).
- [35] The Linux Kernel Documentation. *BPF Type Format (BTF)*. 2026. URL: <https://docs.kernel.org/bpf/btf.html> (visited on 07/27/2026).
- [36] The Linux Kernel Documentation. *Classic BPF vs eBPF*. Describes execution through architecture-specific JIT compilation and, when JIT compilation is not used, through the `__bpf_prog_run()` interpreter. URL: https://docs.kernel.org/bpf/classic_vs_extended.html (visited on 07/27/2026).
- [37] The Linux Kernel Documentation. *Documentation for /proc/sys/net/: BPF JIT Controls*. 2026. URL: <https://docs.kernel.org/admin-guide/sysctl/net.html#bpf-jit-enable> (visited on 07/27/2026).
- [38] The Linux Kernel Documentation. *Kernel Probes (Kprobes)*. 2026. URL: <https://docs.kernel.org/trace/kprobes.html> (visited on 07/27/2026).
- [39] The Linux Kernel Documentation. *libbpf Overview*. 2026. URL: https://docs.kernel.org/bpf/libbpf/libbpf_overview.html (visited on 07/27/2026).
- [40] The Linux Kernel Documentation. *Linux Security Module Development*. 2026. URL: <https://docs.kernel.org/security/lsm-development.html> (visited on 07/27/2026).
- [41] The Linux Kernel Documentation. *Linux Tracing Technologies*. 2026. URL: <https://docs.kernel.org/trace/index.html> (visited on 07/27/2026).

BIBLIOGRAPHY

- [42] The Linux Kernel Documentation. *Perf Ring Buffer*. 2026. URL: https://docs.kernel.org/userspace-api/perf_ring_buffer.html (visited on 07/27/2026).
- [43] The Linux Kernel Documentation. *Program Types and ELF Sections*. 2026. URL: https://docs.kernel.org/bpf/libbpf/program_types.html (visited on 07/27/2026).
- [44] The Linux Kernel Documentation. *Using the Linux Kernel Tracepoints*. 2026. URL: <https://docs.kernel.org/trace/tracepoints.html> (visited on 07/27/2026).
- [45] The Tetragon Authors. *Tetragon Enforcement*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/enforcement/> (visited on 07/27/2026).
- [46] The Tetragon Authors. *Tetragon Events*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/events/> (visited on 07/27/2026).
- [47] The Tetragon Authors. *Tetragon Installation FAQ*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/installation/faq/> (visited on 07/27/2026).
- [48] The Tetragon Authors. *Tetragon Tracing Policies*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/> (visited on 07/27/2026).
- [49] The Tetragon Authors. *Tetragon Tracing Policy Selectors*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/selectors/> (visited on 07/27/2026).

To my family and friends, for their support.